Why Python is not the programming language of the future

Even though it will be in high demand for a few more years

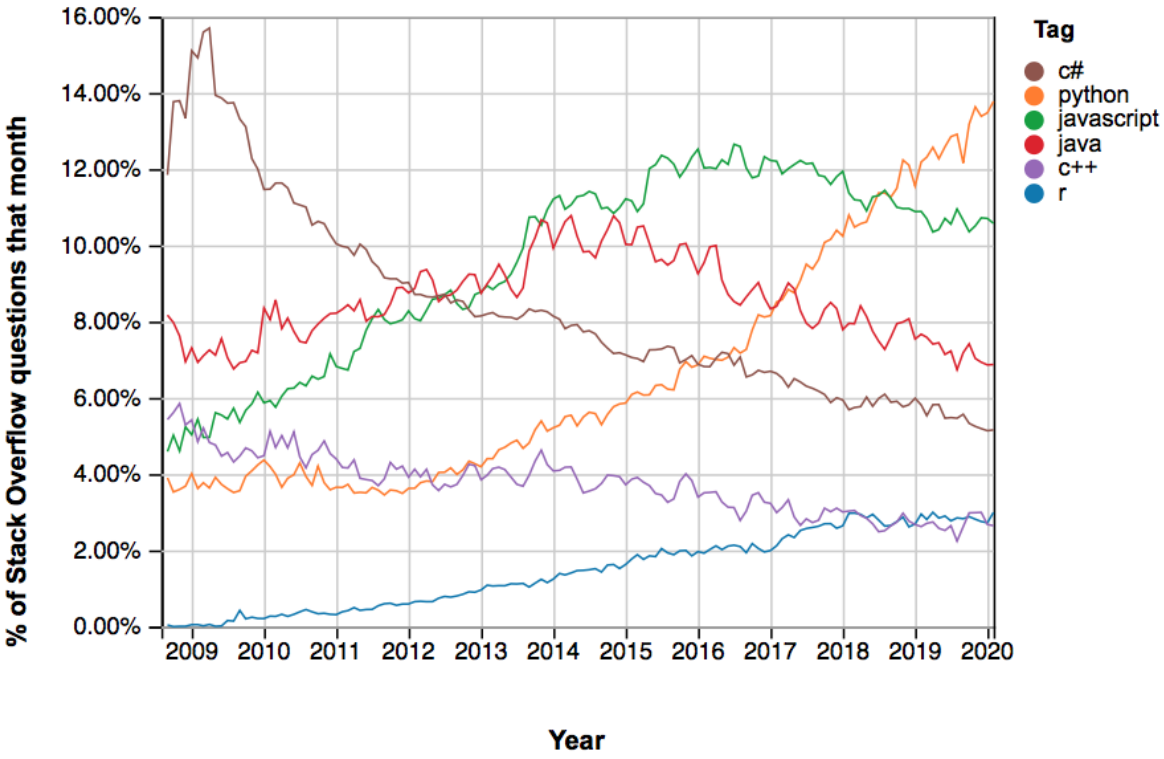
It took the programming community a couple of decades to appreciate Python. But since the early 2010's, it has been booming — and eventually surpassing C, C#, Java and JavaScript in popularity.

But until when will that trend continue? When will Python eventually be replaced by other languages, and why?

Putting an exact expiry date on Python would be so much speculation, it might as well pass as Science-Fiction. Instead, I will assess the virtues that are boosting Python's popularity right now, and the weak points that will break it in the future.

What makes Python popular right now

Python's success is reflected in the Stack Overflow trends, which measure the count of tags in posts on the platform. Given the size of StackOverflow, this is quite a good indicator for language popularity.



Snapshot of tags on various programming languages on StackOverflow.

While R has been plateauing over the last few years, and many other languages are on a steady decline, Python's growth seems unstoppable. Almost 14% of all StackOverflow questions are tagged "python", and the trend is going up. And there are several reasons for that.

It's old

Python has been around since the nineties. That doesn't only mean that it has had plenty of time to grow. It has also acquired a large and supportive community.

So if you have any issue while you're coding in Python, the odds are high that you'll be able to solve it with a single Google search. Simply because somebody will have already encountered your problem and written something helpful about it.

It’s beginner-friendly

It’s not only the fact that it has been around for decades, giving programmers the time to make brilliant tutorials. More than that, the syntax of Python is very human-readable.

For a start, there’s no need to specify the data type. You just declare a variable; Python will understand from the context whether it’s an integer, a float value, a boolean or something else. This is a huge edge for beginners. If you’ve ever had to program in C++, you know how frustrating it is your program won’t compile because you swapped a float for an integer.

And if you’ve ever had to read Python and C++ code side-by-side, you’ll know how understandable Python is. Even though C++ was designed with English in mind, it’s a rather bumpy read compared to Python code.

It’s versatile

Since Python has been around for so long, developers have made a package for every purpose. These days, you can find a package for almost everything.

Want to crunch numbers, vectors and matrices? **NumPy** is your guy.
Want to do calculations for tech and engineering? Use **SciPy**.
Want to go big in data manipulation and analysis? Give **Pandas** a go.
Want to start out with Artificial Intelligence? Why not use **Scikit-Learn**.

Whichever computational task you’re trying to manage, chances are that there is a Python package for it out there. This makes Python stay on top of recent developments, can be seen from the surge in Machine Learning over the past few years.

Downsides of Python — and whether they'll be fatal

Based on the previous elaborations, you could imagine that Python will stay on top of sh*t for ages to come. But like every technology, Python has its weaknesses. I will go through the most important flaws, one by one, and assess whether these are fatal or not.

Speed

Python is slow. Like, really slow. On average, you'll need about 2–10 times longer to complete a task with Python than with any other language.

There are various reasons for that. One of them is that it's dynamically typed — remember that you don't need to specify data types like in other languages. This means that a lot of memory needs to be used, because the program needs to reserve enough space for each variable that it works in any case. And lots of memory usage translates to lots of computing time.

Another reason is that Python can only execute one task at a time. This is a consequence of flexible datatypes — Python needs to make sure each variable has only one datatype, and parallel processes could mess that up.

In comparison, your average web browser can run a dozen different threads at once. And there are some other theories around, too.

But at the end of the day, none of the speed issues matter. Computers and servers have gotten so cheap that we're talking about fractions of

seconds. And the end user doesn't really care whether their app loads in 0.001 or 0.01 seconds.

Scope

Originally, Python was dynamically scoped. This basically means that, to evaluate an expression, a compiler first searches the current block and then successively all the calling functions.

The problem with dynamic scoping is that every expression needs to be tested in every possible context — which is tedious. That's why most modern programming languages use static scoping.

Python tried to transition to static scoping, but messed it up. Usually, inner scopes — for example functions within functions — would be able to see *and* change outer scopes. In Python, inner scopes can only see outer scopes, but not change them. This leads to a lot of confusion.

Lambdas

Despite all of the flexibility within Python, the usage of Lambdas is rather restrictive. Lambdas can only be expressions in Python, and not be statements.

On the other hand, variable declarations and statements are always statements. This means that Lambdas cannot be used for them.

This distinction between expressions and statements is rather arbitrary, and doesn't occur in other languages.

Whitespaces make code more readable, but less maintainable. Photo by Irvan Smith on Unsplash

Whitespaces

In Python, you use whitespaces and indentations to indicate different levels of code. This makes it optically appealing and intuitive to understand.

Other languages, for example C++, rely more on braces and semicolons. While this might not be visually appealing and beginner-friendly, it makes the code a lot more maintainable. For bigger projects, this is a lot more useful.

Newer languages like Haskell solve this problem: They rely on whitespaces, but offer an alternative syntax for those who wish to go without.

Mobile Development

As we're witnessing the shift from desktop to smartphone, it's clear that we need robust languages to build mobile software.

But not many mobile apps are being developed with Python. That doesn't mean that it can't be done — there is a Python package called Kivy for this purpose.

But Python wasn't made with mobile in mind. So even though it might produce passable results for basic tasks, your best bet is to use a language that was created for mobile app development. Some widely used programming frameworks for mobile include React Native, Flutter, Iconic, and Cordova.

To be clear, laptops and desktop computers should be around for many years to come. But since mobile has long surpassed desktop traffic, it's

safe to say that learning Python is not enough to become a seasoned all-round developer.

Runtime Errors

A Python script isn't compiled first and then executed. Instead, it compiles every time you execute it, so any coding error manifests itself at runtime. This leads to poor performance, time consumption, and the need for a lot of tests. Like, *a lot* of tests.

This is great for beginners since testing teaches them a lot. But for seasoned developers, having to debug a complex program in Python makes them go awry. This lack of performance is the biggest factor that sets a timestamp on Python.

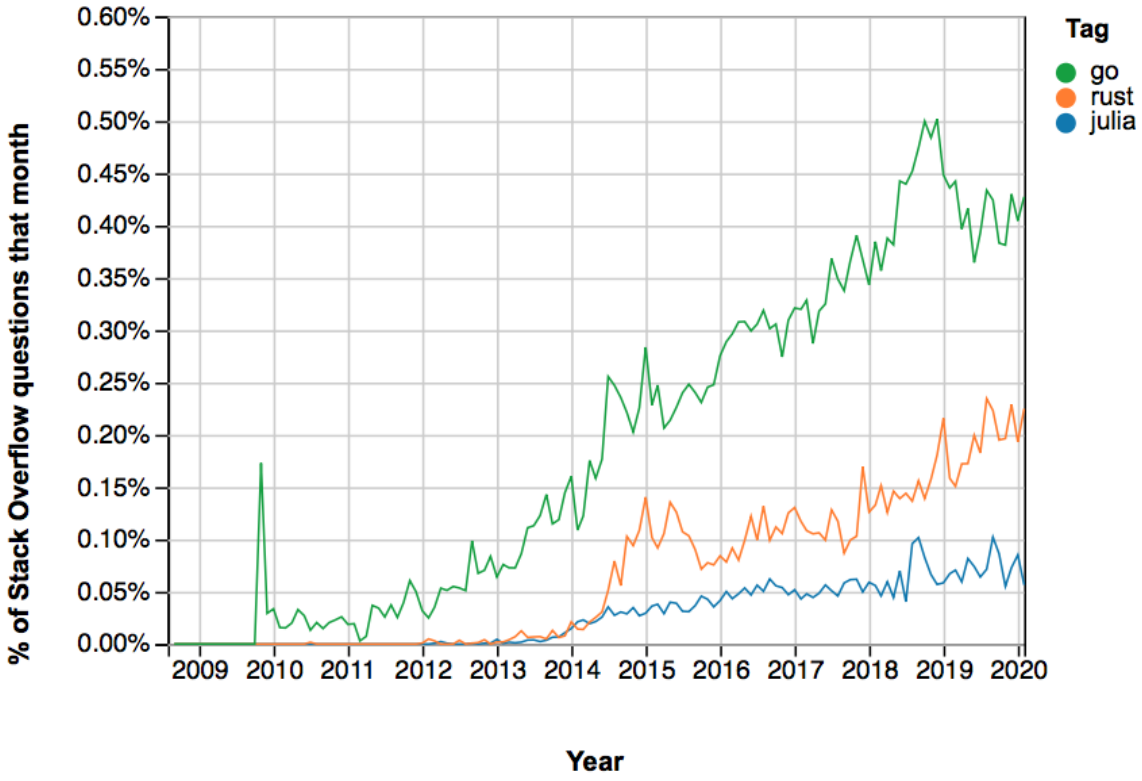
What could replace Python in the future — and when

There are a few new competitors on the market of programming languages:

- **Rust** offers the same kind of safety that Python has — no variable can accidentally be overwritten. But it solves the performance issue with the concept of ownership and borrowing. It is also the most-loved programming language of the last few years, according to StackOverflow Insights.
- **Go** is great for beginners like Python. And it is so simple that it's even easier to maintain the code. Fun point: Go developers are among the highest-paid programmers on the market.

- **Julia** is a very new language that competes head-on with Python. It fills the gap of large-scale technical computations: Usually, one would have used Python or Matlab, and patched the whole thing up with C++ libraries, which are necessary at a large scale. Now, one can use Julia instead of juggling with two languages.

While there are other languages on the market, Rust, Go, and Julia are the ones that fix weak patches of Python. All of these languages excel in yet-to-come technologies, most notably in Artificial Intelligence. While their market share is still small, as reflected in the number of StackOverflow tags, the trend for all of them is clear: upwards.



Snapshot of tags on various programming languages on StackOverflow.

Given the ubiquitous popularity of Python at the moment, it will surely take half a decade, maybe even a whole, for any of these new languages to replace it.

Which of the languages it will be — Rust, Go, Julia, or a new language of the future — is hard to say at this point. But given the performance issues that are fundamental in the architecture of Python, one will inevitably take its spot.